ÉCOLE 42 · TRONC COMMUN · SUJET V19.3

Libft

Ta première bibliothèque C : 43 fonctions, un Makefile et un README, à rendre sans crash ni fuite mémoire.

23	11	9
fonctions de la libc	fonctions additionnelles	fonctions de liste chaînée

LE CONTEXTE

Qu’est-ce que Libft ?

Une bibliothèque statique. libft.a réunit des fonctions que tu réécris toi-même.

Un outil pour la suite. Le sujet la présente comme utile pour tes futurs projets d’école en C.

Trois familles. Des fonctions de la libc, des fonctions additionnelles et une bibliothèque de listes chaînées.



Chaque .c est compilé avec les flags exigés, puis archivé avec ar : libtool est interdit.

LE CONTEXTE

Pourquoi 42 donne ce projet

Comprendre la libc

Réécrire strlen, memmove ou atoi montre ce que ces fonctions font vraiment.

Maîtriser les bases

Pointeurs, chaînes, malloc et free, pointeurs de fonctions.

Gagner en rigueur

Norme, zéro crash, zéro fuite : des habitudes de développeur.

Apprendre l’outillage

Makefile, archive avec ar, dépôt Git, tests personnels.

Une boîte à outils

Réutilisable dans les projets suivants, copiée dans un dossier libft.

Expliquer son code

L’évaluation par les pairs vérifie que tu comprends ce que tu rends.

Règles communes du sujet

Règle	Ce que le sujet exige
Langage	Le projet est écrit en C
Norme	Respect de la Norme ; une erreur donne 0, bonus compris
Stabilité	Ni segfault, ni bus error, ni double free (hors comportement indéfini), sinon 0
Mémoire	Toute mémoire allouée est libérée ; aucune fuite tolérée
Compilation	cc avec -Wall -Wextra -Werror, sans relink inutile
Makefile	$(NAME), all, clean, fclean, re (et bonus si tu rends des bonus)
Tests	Encouragés, mais non rendus et non notés
Rendu	Dépôt Git assigné : seul son contenu est évalué

LES RÈGLES

Contraintes techniques

Sujet	Contrainte
Variables globales	Strictement interdites
Fonctions auxiliaires	Déclarées static pour limiter leur portée au fichier
Emplacement	Tous les fichiers à la racine du dépôt
Fichiers inutiles	Soumettre des fichiers inutilisés n’est pas autorisé
Compilation	Chaque .c compile avec -Wall -Wextra -Werror
Archive	La commande ar est obligatoire, libtool est interdit
libft.a	Créée à la racine du dépôt
restrict et C99	Interdits dans tes prototypes, comme le flag -std=c99

LES RÈGLES

Ce qu’il faut rendre

Élément	Valeur du sujet
Programme	libft.a
Fichiers à rendre	Makefile, libft.h, ft_*.c
Règles du Makefile	NAME, all, clean, fclean, re
Fonctions externes	Précisées fonction par fonction : malloc, free, write
Libft autorisée	Sans objet pour ce projet

> Place tous les fichiers à la racine du dépôt et vérifie le nom de chacun avant la défense.

LE CONTENU

Trois parties, 43 fonctions

PARTIE 1

23

Fonctions de la libc

Même prototype et même comportement que l'original, avec le préfixe ft_.

PARTIE 2

11

Fonctions additionnelles

Absentes de la libc ou existant sous une autre forme.

PARTIE 3

9

Listes chaînées

La structure t_list et les fonctions pour la manipuler.

> Les fonctions de la partie 1 peuvent servir à écrire celles de la partie 2.

PARTIE 1

Les 23 fonctions de la libc

CARACTÈRES ET NOMBRES

- ft_isalpha
- ft_isdigit
- ft_isalnum
- ft_isascii
- ft_isprint
- ft_toupper
- ft_tolower
- ft_atoi

MÉMOIRE

- ft_memset
- ft_bzero
- ft_memcpy
- ft_memmove
- ft_memchr
- ft_memcmp
- ft_calloc

CHAÎNES

- ft_strlen
- ft_strlcpy
- ft_strlcat
- ft_strchr
- ft_strrchr
- ft_strncmp
- ft_strnstr
- ft_strdup

Mêmes prototypes et mêmes comportements que la page man, seul le préfixe ft_ change.

PARTIE 1

Les pièges de la libc

Fonction	Point d’attention
memcpy, memmove	memcpy suppose des zones sans chevauchement ; memmove les gère en choisissant le sens de copie
strlcpy, strlcat	Copient au plus size - 1 caractères, terminent par '\0' et retournent la longueur totale qu’elles ont tenté de créer
strchr, strrchr	Chercher '\0' retourne un pointeur vers la fin de la chaîne
strncmp, memcmp	Comparaison en unsigned char ; memcmp ne s’arrête pas au '\0'
calloc	Si nmemb ou size vaut 0, retourne un pointeur unique que free() accepte
strnstr	Une aiguille vide retourne la meule de foin ; la recherche est limitée à len caractères
is*	Retournent 1 si le caractère correspond, 0 sinon
Tests sous glibc	strlcpy, strlcat et bzero : inclure <bsd/string.h> et compiler avec -lbsd

PARTIE 2

Les 11 fonctions additionnelles

Fonction	Rôle	Retour
ft_substr	Sous-chaîne depuis start, de longueur max len	NULL si échec
ft_strjoin	Concatène s1 et s2 dans une nouvelle chaîne	NULL si échec
ft_strtrim	Copie de s1 sans les caractères de set aux extrémités	NULL si échec
ft_split	Découpe s selon le séparateur c	Tableau, ou NULL
ft_itoa	Convertit un entier, négatifs compris	NULL si échec
ft_strmapi	Applique f(index, char), crée une nouvelle chaîne	NULL si échec
ft_striteri	Applique f(index, adresse) en place	Aucun

Écriture sur un descripteur. ft_putchar_fd, ft_putstr_fd, ft_putendl_fd et ft_putnbr_fd écrivent avec write, seule fonction externe autorisée.

PARTIE 2

Les cas limites à connaître

`ft_substr`

Si start dépasse la longueur de s, on retourne une chaîne vide allouée. Si len dépasse le reste, on tronque.

`ft_split`

Les séparateurs consécutifs ne créent pas de chaîne vide. En cas d'échec, on libère les chaînes déjà allouées, puis le tableau.

`ft_itoa`

Trois cas : INT_MIN, 0 et les négatifs. Pour INT_MIN, écrire -n en int déborde.

`ft_putnbr_fd`

Même piège que ft_itoa avec INT_MIN, et write comme seule fonction externe.

PARTIE 3

Les 9 fonctions de liste chaînée

```
typedef struct s_list
{
    void *content;
    struct s_list *next;
} t_list;
```

Créer et ajouter. ft_lstnew, ft_lstadd_front, ft_lstadd_back

Parcourir. ft_lstsize, ft_lstlast, ft_lstiter

Libérer et transformer. ft_lstdelone, ft_lstclear, ft_lstmap

Deux idées clés. t_list ** quand la tête peut changer, et une fonction del pour libérer un contenu de type inconnu.

RENDRE

Un Makefile conforme au sujet

```
NAME = libft.a
CC = cc
CFLAGS = -Wall -Wextra -Werror
SRCS = ft_isalpha.c ft_strlen.c ft_split.c
OBJS = $(SRCS:.c=.o)

all: $(NAME)

$(NAME): $(OBJS)
    ar rcs $(NAME) $(OBJS)

%.o: %.c libft.h
    $(CC) $(CFLAGS) -c $< -o $@
```

```
clean:
    rm -f $(OBJS)

fclean: clean
    rm -f $(NAME)

re: fclean all

.PHONY: all clean fclean re
```

SRCS doit lister tous tes fichiers ft_*.c.

Les .o dépendent du header : pas de relink inutile.

Chaque recette commence par une tabulation, pas des espaces.